



# Chapter 1: Introduction

**Database System Concepts, 6<sup>th</sup> Ed.**

©Silberschatz, Korth and Sudarshan

See [www.db-book.com](http://www.db-book.com) for conditions on re-use



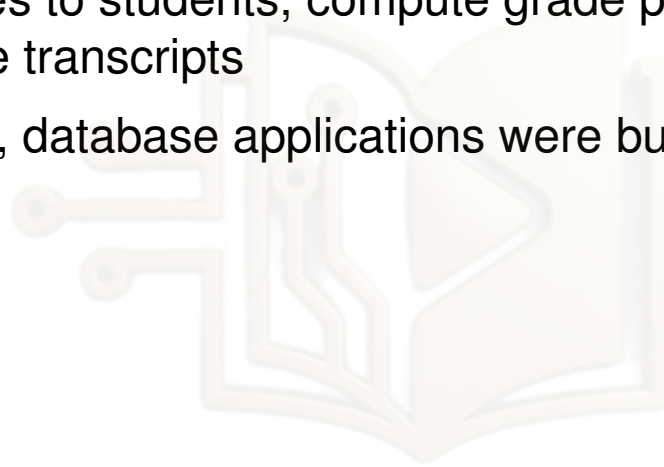
# Database Management System (DBMS)

- DBMS contains information about a particular enterprise
  - Collection of interrelated data
  - Set of programs to access the data
  - An environment that is both *convenient* and *efficient* to use
- Database Applications:
  - Banking: transactions
  - Airlines: reservations, schedules
  - Universities: registration, grades
  - Sales: customers, products, purchases
  - Online retailers: order tracking, customized recommendations
  - Manufacturing: production, inventory, orders, supply chain
  - Human resources: employee records, salaries, tax deductions
- Databases can be very large.
- Databases touch all aspects of our lives



# University Database Example

- Application program examples
  - Add new students, instructors, and courses
  - Register students for courses, and generate class rosters
  - Assign grades to students, compute grade point averages (GPA) and generate transcripts
- In the early days, database applications were built directly on top of file systems





# Drawbacks of using file systems to store data

- Data redundancy and inconsistency
  - ▶ Multiple file formats, duplication of information in different files
- Difficulty in accessing data
  - ▶ Need to write a new program to carry out each new task
- Data isolation — multiple files and formats
- Integrity problems
  - ▶ Integrity constraints (e.g., account balance  $> 0$ ) become “buried” in program code rather than being stated explicitly
  - ▶ Hard to add new constraints or change existing ones



# Drawbacks of using file systems to store data (Cont.)

- Atomicity of updates
  - ▶ Failures may leave database in an inconsistent state with partial updates carried out
  - ▶ Example: Transfer of funds from one account to another should either complete or not happen at all
- Concurrent access by multiple users
  - ▶ Concurrent access needed for performance
  - ▶ Uncontrolled concurrent accesses can lead to inconsistencies
    - Example: Two people reading a balance (say 100) and updating it by withdrawing money (say 50 each) at the same time
- Security problems
  - ▶ Hard to provide user access to some, but not all, data

**Database systems offer solutions to all the above problems**



# Levels of Abstraction

- **Physical level:** describes how a record (e.g., customer) is stored.
- **Logical level:** describes data stored in database, and the relationships among the data.

**type** *instructor* = **record**

```
ID : string;  
name : string;  
dept_name : string;  
salary : integer;
```

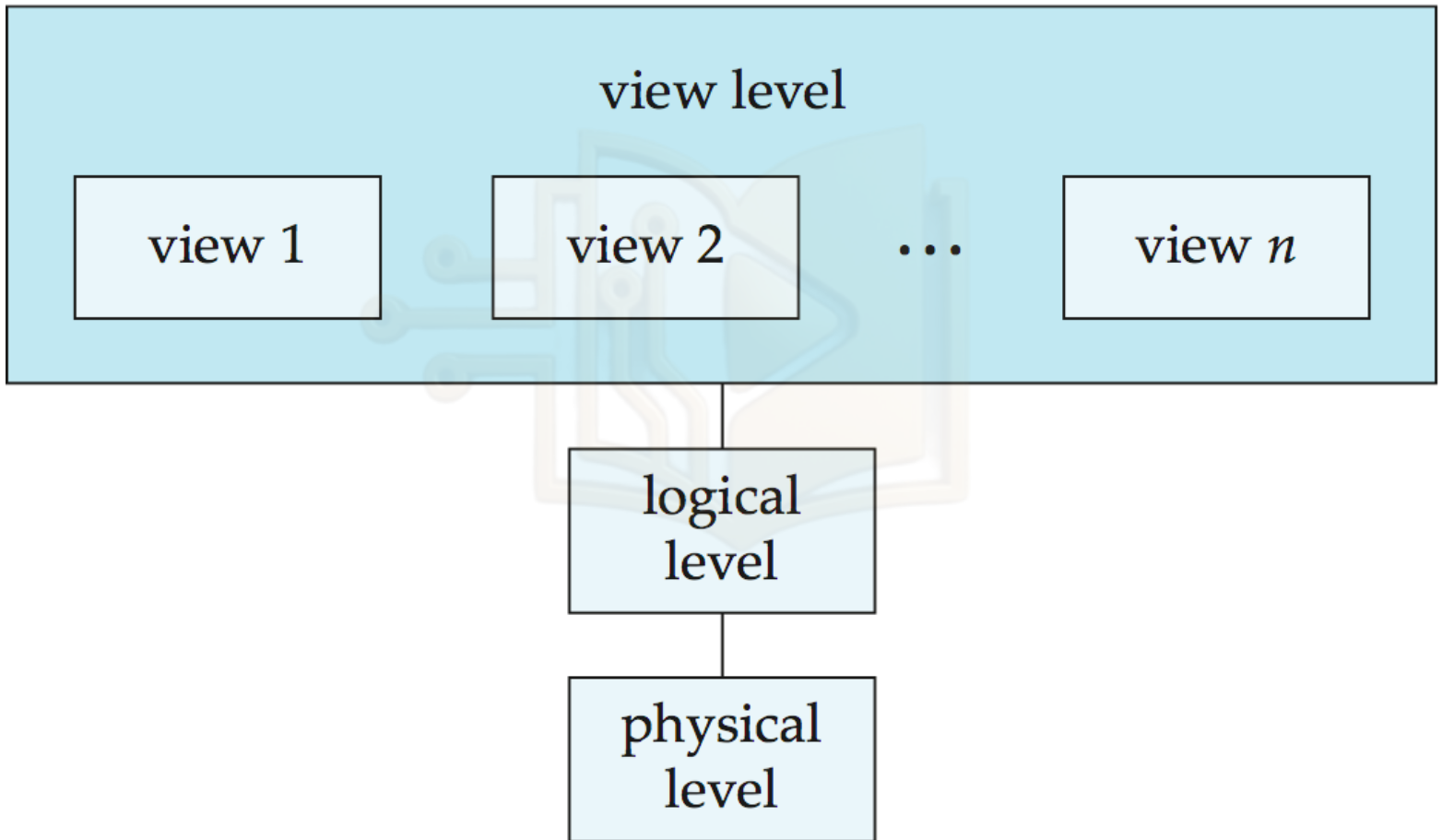
**end;**

- **View level:** application programs hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.



# View of Data

An architecture for a database system





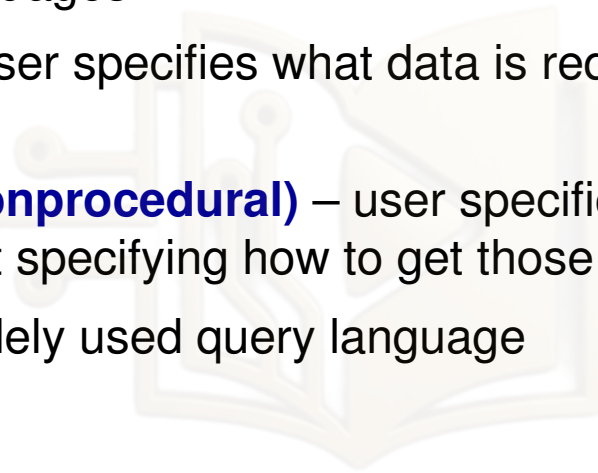
# Instances and Schemas

- Similar to types and variables in programming languages
- **Schema** – the logical structure of the database
  - Example: The database consists of information about a set of customers and accounts and the relationship between them
  - Analogous to type information of a variable in a program
  - **Physical schema**: database design at the physical level
  - **Logical schema**: database design at the logical level
- **Instance** – the actual content of the database at a particular point in time
  - Analogous to the value of a variable
- **Physical Data Independence** – the ability to modify the physical schema without changing the logical schema
  - Applications depend on the logical schema
  - In general, the interfaces between the various levels and components should be well defined so that changes in some parts do not seriously influence others.



# Data Manipulation Language (DML)

- Language for accessing and manipulating the data organized by the appropriate data model
  - DML also known as query language
- Two classes of languages
  - **Procedural** – user specifies what data is required and how to get those data
  - **Declarative (nonprocedural)** – user specifies what data is required without specifying how to get those data
- SQL is the most widely used query language





# Data Definition Language (DDL)

- Specification notation for defining the database schema

Example:        **create table** *instructor* (  
                              *ID*              **char**(5),  
                              *name*          **varchar**(20),  
                              *dept\_name* **varchar**(20),  
                              *salary*      **numeric**(8,2))

- DDL compiler generates a set of table templates stored in a ***data dictionary***
- Data dictionary contains metadata (i.e., data about data)
  - Database schema
  - Integrity constraints
    - ▶ Primary key (ID uniquely identifies instructors)
    - ▶ Referential integrity (**references** constraint in SQL)
      - e.g. *dept\_name* value in any *instructor* tuple must appear in *department* relation
  - Authorization



# SQL

## ■ **SQL**: widely used non-procedural language

- Example: Find the name of the instructor with ID 22222

```
select   name  
from     instructor  
where    instructor.ID = '22222'
```

- Example: Find the ID and building of instructors in the Physics dept.

```
select instructor.ID, department.building  
from   instructor, department  
where  instructor.dept name = "physics"
```

## ■ Application programs generally access databases through one of

- Language extensions to allow embedded SQL
- Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database

## ■ Chapters 3, 4 and 5



# Database Design

The process of designing the general structure of the database:

- Logical Design – Deciding on the database schema. Database design requires that we find a “good” collection of relation schemas.
  - Business decision – What attributes should we record in the database?
  - Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?
- Physical Design – Deciding on the physical layout of the database



# History of Database Systems

- 1950s and early 1960s:
  - Data processing using magnetic tapes for storage
    - ▶ Tapes provided only sequential access
  - Punched cards for input
- Late 1960s and 1970s:
  - Hard disks allowed direct access to data
  - Network and hierarchical data models in widespread use
  - Ted Codd defines the relational data model
    - ▶ Would win the ACM Turing Award for this work
    - ▶ IBM Research begins System R prototype
    - ▶ UC Berkeley begins Ingres prototype
  - High-performance (for the era) transaction processing



# History (cont.)

- 1980s:
  - Research relational prototypes evolve into commercial systems
    - ▶ SQL becomes industrial standard
  - Parallel and distributed database systems
  - Object-oriented database systems
- 1990s:
  - Large decision support and data-mining applications
  - Large multi-terabyte data warehouses
  - Emergence of Web commerce
- Early 2000s:
  - XML and XQuery standards
  - Automated database administration
- Later 2000s:
  - Giant data storage systems
    - ▶ Google BigTable, Yahoo PNuts, Amazon, ..



# End of Chapter 1

